📐 MATHEMATICS FOR MACHINE LEARNING

CampusX · Data Science Mentorship Program 2023 · Video 5 of 23

NORMAL DISTRIBUTION

Session 41 — The Gaussian Bell Curve · Z-Scores · Empirical Rule · QQ Plots

Bell Curve Equation · μ and σ Parameters · 68-95-99.7 Rule · Standard Normal · Z-Score & Z-Table · QQ Plot · Normality Tests · Log-Normal

Session Notes — Complete Reference

Playlist: Maths for Machine Learning · Instructor: Nitish Singh (CampusX)

📑 Table of Contents

1 · Why Normal Distribution Matters

The **Normal Distribution** (Gaussian Distribution) is arguably the most important probability distribution in all of statistics, data science, and machine learning. Understanding it deeply is non-negotiable.

1.1 Why Is It Everywhere?

- **Nature follows it:** Heights, weights, blood pressure, IQ scores, measurement errors — all tend to be normally distributed
- **Central Limit Theorem:** The mean of any sufficiently large random sample will be approximately normally distributed, regardless of the underlying distribution
- **Mathematical convenience:** Many statistical methods are derived under the assumption of normality
- **ML algorithms depend on it:** Linear Regression (residuals), Logistic Regression, LDA, Gaussian Naive Bayes, Gaussian Mixture Models all assume normality

> 💡 **The "Normal" Name**
>
> It's called "normal" not because other distributions are "abnormal," but because it was historically considered the norm for describing natural variation. It is also called the **Gaussian Distribution** after Carl Friedrich Gauss, who used it to model astronomical errors.

2 · The Bell Curve — Equation & Parameters

2.1 The PDF Formula

NORMAL DISTRIBUTION PDF

$f(x) = (1 / \sigma\sqrt{(2\pi)}) \cdot e^{[-(x - \mu)^2 / (2\sigma^2)]}$ where: μ = mean (centre of the bell curve) σ = standard deviation (width/spread) σ^2 = variance e = Euler's number ≈ 2.71828 $\pi \approx 3.14159$

2.2 Breaking Down the Formula

COMPONENT	PURPOSE	EFFECT
$1 / \sigma\sqrt{(2\pi)}$	Normalisation constant	Ensures total area under curve = 1
$(x - \mu)^2$	Squared distance from mean	Makes the curve symmetric about μ
$-(x - \mu)^2 / (2\sigma^2)$	Negative exponent	Values far from μ get exponentially smaller density
e^[…]	Exponential function	Creates the smooth bell shape

2.3 Notation

STANDARD NOTATION

$X \sim N(\mu, \sigma^2)$ "X follows a Normal distribution with mean μ and variance σ^2" Examples: $X \sim N(0, 1)$ Standard Normal Distribution $X \sim N(100, 225)$ IQ scores ($\mu=100$, $\sigma=15$, $\sigma^2=225$)

3 · Properties of the Normal Distribution

#	PROPERTY	DETAILS
1	**Bell-shaped & Symmetric**	Perfectly symmetric about the mean μ
2	**Mean = Median = Mode**	All three measures of central tendency are identical
3	**Unimodal**	Exactly one peak (the mode) at $x = \mu$
4	**Asymptotic**	Tails approach zero but never touch the x-axis (extends to $\pm\infty$)
5	**Total area = 1**	$\int_{-\infty}^{+\infty} f(x)\,dx = 1$
6	**Defined by two parameters**	μ (location) and σ (scale) completely specify the distribution
7	**Skewness = 0**	No skew — perfectly balanced left and right
8	**Kurtosis = 3 (excess = 0)**	The baseline "mesokurtic" distribution
9	**Inflection points**	At $x = \mu \pm \sigma$ (where the curve changes from concave to convex)
10	**Additive property**	Sum of independent normals is also normal: $X + Y \sim N(\mu_X+\mu_Y, \sigma_X^2+\sigma_Y^2)$

📊 Anatomy of the Bell Curve

```
                      Peak = f(μ) = 1/(σ√2π)
                             ▼
                          ┌──*──┐
                         /  │   \          ← Inflection points at μ±σ
                       /    │     \
                     /      │       \
                  /─────────│─────────\
                /           │           \
   ──/──────────────────────│──────────────────────\──── x-axis
     │    │    │            μ         │    │    │
   μ-3σ μ-2σ  μ-σ                    μ+σ  μ+2σ  μ+3σ

     ←── 68.27% ──→                  (1σ)
     ←─────── 95.45% ───────→        (2σ)
     ←──────────── 99.73% ────────────→    (3σ)
```

4 · Empirical Rule (68-95-99.7)

The **Empirical Rule** (also called the 3–sigma rule) is the most practically useful property of the normal distribution. It tells you exactly how much data falls within 1, 2, or 3 standard deviations of the mean.

THE 68-95-99.7 RULE

$P(\mu - 1\sigma \le X \le \mu + 1\sigma) = 68.27\%$ ← About 2/3 of data $P(\mu - 2\sigma \le X \le \mu + 2\sigma) = 95.45\%$ ← Almost all data $P(\mu - 3\sigma \le X \le \mu + 3\sigma) = 99.73\%$ ← Virtually all data

📝 **Example — IQ Scores ($\mu = 100, \sigma = 15$)**

- **68%** of people have IQ between **85 and 115** (100 ± 15)
- **95%** of people have IQ between **70 and 130** (100 ± 30)
- **99.7%** of people have IQ between **55 and 145** (100 ± 45)

An IQ above 145 is in the top 0.15% of the population!

4.1 Practical Uses of the Empirical Rule

- **Outlier detection:** Data beyond 3σ from the mean is almost certainly an outlier (~0.27% chance)
- **Quality control:** Six Sigma methodology (3σ on each side = 99.99966%)
- **Quick estimation:** Rapidly estimate probabilities without needing a Z-table
- **Data validation:** If data doesn't follow 68–95–99.7, it may not be normally distributed

5 · Standard Normal Distribution & Z-Score

5.1 The Standard Normal Distribution

The **Standard Normal Distribution** is a special normal distribution with **$\mu = 0$** and **$\sigma = 1$**. It is denoted as **$Z \sim N(0, 1)$**.

> 💡 **Why Standardize?**
>
> Every normal distribution has a different μ and σ. To compare values across different distributions (e.g., comparing a score on one exam to a score on a different exam), we transform everything to the **same standard scale**. One Z-table works for ALL normal distributions.

5.2 The Z-Score Formula

Z-SCORE (STANDARDIZATION)

```
Z = (X - μ) / σ where: X = raw value μ = mean of the distribution σ = standard deviation Z = number of standard deviations from the mean
```

5.3 Interpreting the Z-Score

Z-SCORE	MEANING	PERCENTILE (APPROX)
Z = 0	Exactly at the mean	50th
Z = +1	1 SD above the mean	84.13th
Z = +2	2 SD above the mean	97.72nd
Z = +3	3 SD above the mean	99.87th
Z = -1	1 SD below the mean	15.87th
Z = -2	2 SD below the mean	2.28th
Z = -3	3 SD below the mean	0.13th

📝 **Example — Comparing Exam Scores**

Exam A: Score = 85, $\mu = 70$, $\sigma = 10$ → $Z = (85-70)/10 =$ **+1.50**
Exam B: Score = 92, $\mu = 80$, $\sigma = 5$ → $Z = (92-80)/5 =$ **+2.40**

Although 92 > 85, the Z-scores reveal that the Exam B score is **more exceptional** (2.4 SD above mean vs 1.5 SD). The student performed relatively better on Exam B.

6 · Using Z-Tables

A **Z-table** maps Z-scores to cumulative probabilities (area under the standard normal curve to the left of Z).

6.1 How to Read a Z-Table

1. Compute the Z-score: $Z = (X - \mu) / \sigma$
2. Find the row matching the integer + first decimal (e.g., 1.5 for Z = 1.53)
3. Find the column matching the second decimal (e.g., 0.03)
4. The intersection gives $P(Z \leq 1.53)$

6.2 Common Probability Calculations

Z-TABLE PROBABILITY FORMULAS

```
P(Z ≤ z) = Read directly from Z-table = Φ(z) P(Z > z) = 1 - Φ(z) P(Z ≤ -z) = 1 - Φ(z) (by symmetry) P(a ≤ Z ≤ b) = Φ(b) - Φ(a)
```

📝 **Example — Finding Probabilities**

Problem: Heights are N(170, 25) — μ=170 cm, σ=5 cm. Find P(height > 180 cm).

Step 1: $Z = (180 - 170) / 5 = +2.0$
Step 2: From Z-table, $\Phi(2.0) = 0.9772$
Step 3: $P(X > 180) = 1 - 0.9772 =$ **0.0228 = 2.28%**

Only about 2.3% of people are taller than 180 cm in this population.

💡 **In Python — No Z-Table Needed!**

Use `scipy.stats.norm.cdf(x, μ, σ)` directly. It computes the exact probability without looking up tables. The Z-table is for conceptual understanding and exams.

7 · CDF of the Normal Distribution

The CDF of the normal distribution, denoted Φ(x), gives the probability that a normal random variable is less than or equal to x.

NORMAL CDF

$\Phi(x) = P(X \leq x) = \int_{-\infty}^{x} f(t)\,dt$ No closed-form solution! Must be computed numerically. That's why Z-tables and computer functions exist.

7.1 Key CDF Values to Remember

X (IN Z-SCORES)	Φ(X) = P(Z ≤ X)	P(Z > X)
−3.0	0.0013	0.9987
−2.0	0.0228	0.9772
−1.0	0.1587	0.8413
0	**0.5000**	**0.5000**
+1.0	0.8413	0.1587
+1.645	0.9500	0.0500 (← 5% significance!)
+1.96	0.9750	0.0250 (← 2.5% two-tailed)
+2.0	0.9772	0.0228
+2.576	0.9950	0.0050 (← 1% significance)
+3.0	0.9987	0.0013

8 · Effect of μ and σ on the Curve

8.1 Changing μ (Mean) — Shifts the Curve

- Increasing μ → Curve shifts **right**
- Decreasing μ → Curve shifts **left**
- Shape and spread remain unchanged

8.2 Changing σ (Standard Deviation) — Changes Width

- Small σ → **Tall and narrow** curve (data concentrated near mean)
- Large σ → **Short and wide** curve (data spread out)
- Peak height = $1/(\sigma\sqrt{2\pi})$ — smaller σ means taller peak
- Total area always remains 1

📊 N(0, 1) vs N(5, 1)

Same shape, different centre. Changing μ only **slides** the curve along x-axis without changing its shape.

📊 N(0, 1) vs N(0, 4)

Same centre, different spread. σ=2 is twice as wide as σ=1. The wider curve is shorter because total area must stay = 1.

9 · Checking Normality — QQ Plot & Statistical Tests

Before applying methods that assume normality, you must **verify** that your data is approximately normally distributed.

9.1 QQ Plot (Quantile-Quantile Plot)

The **QQ plot** is the most popular graphical method for assessing normality. It plots the quantiles of your data against the quantiles of a theoretical normal distribution.

How to Read a QQ Plot

PATTERN	INTERPRETATION
Points lie on the **45° diagonal line**	Data is **normally distributed** ✅
Points curve **upward at both ends** (S-shape)	**Heavy tails** (leptokurtic) — more extreme values than expected
Points curve **downward at both ends**	**Light tails** (platykurtic) — fewer extreme values
Points curve **upward at right end**	**Right-skewed** (positive skew)
Points curve **upward at left end**	**Left-skewed** (negative skew)

9.2 Statistical Tests for Normality

TEST	BEST FOR	H_0	PYTHON
Shapiro-Wilk	Small to medium samples (n < 5000)	Data is normal	`stats.shapiro(data)`
D'Agostino-Pearson	Medium to large samples	Data is normal	`stats.normaltest(data)`
Kolmogorov-Smirnov	Comparing to any distribution	Data follows specified dist	`stats.kstest(data, 'norm')`
Anderson-Darling	Emphasis on tail behaviour	Data is normal	`stats.anderson(data)`

⚠️ p-value Interpretation

- p-value > 0.05 → **Fail to reject H_0** → Data is consistent with normality
- p-value ≤ 0.05 → **Reject H_0** → Data is significantly non-normal

Caution: With very large samples, even tiny deviations from normality become statistically significant. Always use both **visual (QQ plot)** and **statistical tests** together.

10 · Skewness & Kurtosis Revisited

The normal distribution serves as the baseline reference for measuring skewness and kurtosis.

10.1 Skewness

TYPE	SKEWNESS VALUE	SHAPE	MEAN VS MEDIAN
Symmetric (Normal)	= 0	Perfectly balanced	Mean = Median = Mode
Right-skewed (Positive)	> 0	Long tail to the right	Mean > Median > Mode
Left-skewed (Negative)	< 0	Long tail to the left	Mean < Median < Mode

10.2 Kurtosis

TYPE	EXCESS KURTOSIS	TAIL BEHAVIOUR	PEAK
Mesokurtic (Normal)	= 0 (kurtosis = 3)	Normal tails — baseline	Moderate peak
Leptokurtic	> 0	Heavier/fatter tails, more outliers	Sharper peak
Platykurtic	< 0	Lighter/thinner tails, fewer outliers	Flatter peak

💡 **Rule of Thumb**

For data to be considered "approximately normal":

- |Skewness| < **0.5** → acceptable
- |Skewness| between 0.5 and 1 → moderately skewed
- |Skewness| > 1 → highly skewed, consider transformation

11 · Log-Normal Distribution & Data Transformations

11.1 Log-Normal Distribution

A variable X follows a **log-normal distribution** if **ln(X)** is normally distributed. It's extremely common in real-world data.

LOG-NORMAL RELATIONSHIP

```
If X ~ LogNormal → ln(X) ~ Normal If Y ~ Normal → e^Y ~ LogNormal
```

Real-World Examples of Log-Normal Data

- **Income/salary** distributions
- **Stock prices** and returns
- **City populations**
- **File sizes** on a computer
- **Duration** of events (hospital stays, phone calls)

11.2 Common Transformations

TRANSFORMATION	FORMULA	BEST FOR	PYTHON
Log Transform	X' = log(X)	Right-skewed data, log-normal data	`np.log(data)`
Square Root	X' = √X	Count data, moderate skew	`np.sqrt(data)`
Box-Cox	X' = (X^λ – 1)/λ	Finds optimal λ automatically	`stats.boxcox(data)`
Yeo-Johnson	Modified Box-Cox	Handles negative values too	`PowerTransformer()`

⚠️ When to Transform

Transform your data if the ML algorithm **assumes normality** (e.g., Linear Regression residuals). Tree-based models (Random Forest, XGBoost) do NOT need normal data — transformations won't help and may hurt interpretability.

12 · Applications in Machine Learning

APPLICATION	HOW NORMAL DISTRIBUTION IS USED
Feature Scaling	StandardScaler transforms features to Z-scores: $Z = (X-\mu)/\sigma$
Outlier Detection	Points beyond $\mu \pm 3\sigma$ flagged as outliers
Gaussian Naive Bayes	Assumes features are normally distributed within each class
Linear Regression	Assumes residuals (errors) are normally distributed
Hypothesis Testing	Z-tests and t-tests are built on normal distribution
Confidence Intervals	$CI = \bar{X} \pm Z^*(\sigma/\sqrt{n})$ — relies on normal sampling distribution
Gaussian Mixture Models	Models data as mixture of multiple normal distributions
Weight Initialization	Neural network weights often initialized from $N(0, \sigma^2)$
Batch Normalization	Normalizes layer inputs to $N(0, 1)$ during training
Generative Models	VAEs sample from $N(0, 1)$ latent space

13 · Python Implementation — Complete Code

13.1 Visualizing the Bell Curve

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# ── Plot multiple normal distributions ──
x = np.linspace(-10, 15, 1000)
params = [(0, 1, 'N(0,1)'), (0, 2, 'N(0,4)'), (3, 1, 'N(3,1)'), (5, 3, 'N(5,9)')]
colors = ['#3b82f6', '#ef4444', '#10b981', '#f59e0b']

plt.figure(figsize=(10, 5))
for (mu, sigma, label), color in zip(params, colors):
    plt.plot(x, stats.norm.pdf(x, mu, sigma), label=label, color=color, lw=2)
plt.title('Effect of μ and σ on Normal Distribution', fontsize=14, fontweight='bold')
plt.xlabel('x'); plt.ylabel('f(x)')
plt.legend(); plt.grid(alpha=0.3); plt.tight_layout()
plt.show()
```

13.2 Empirical Rule Visualization

```python
mu, sigma = 0, 1
x = np.linspace(-4, 4, 1000)
y = stats.norm.pdf(x, mu, sigma)

fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(x, y, 'k', lw=2)

# Fill 1σ, 2σ, 3σ regions
for n_sigma, color, alpha in [(3, '#dbeafe', 1), (2, '#93c5fd', 1), (1, '#3b82f6', 0.5)]:
    x_fill = np.linspace(-n_sigma, n_sigma, 500)
    ax.fill_between(x_fill, stats.norm.pdf(x_fill, mu, sigma), color=color, alpha=alpha)

ax.set_title('Empirical Rule: 68-95-99.7', fontsize=14, fontweight='bold')
ax.set_xlabel('Standard Deviations from Mean'); ax.set_ylabel('Density')
ax.annotate('68.27%', xy=(0, 0.15), fontsize=14, ha='center', fontweight='bold', color='white')
plt.tight_layout(); plt.show()
```

13.3 Z-Score & Probability Calculation

```
# ── Z-Score Calculation ──
x_val, mu, sigma = 85, 70, 10
z_score = (x_val - mu) / sigma
print(f"Z-score for X={x_val}: {z_score:.2f}")

# ── Probability using CDF ──
p_below = stats.norm.cdf(x_val, mu, sigma)
p_above = 1 - p_below  # or stats.norm.sf(x_val, mu, sigma)
p_between = stats.norm.cdf(80, mu, sigma) - stats.norm.cdf(60, mu, sigma)

print(f"P(X ≤ {x_val}) = {p_below:.4f}")
print(f"P(X > {x_val}) = {p_above:.4f}")
print(f"P(60 ≤ X ≤ 80) = {p_between:.4f}")

# ── Inverse CDF (find X for a given percentile) ──
x_90th = stats.norm.ppf(0.90, mu, sigma)
print(f"90th percentile = {x_90th:.2f}")
```

13.4 QQ Plot & Normality Test

```
from scipy.stats import probplot, shapiro
import seaborn as sns

# Generate data
np.random.seed(42)
normal_data = np.random.normal(50, 10, 500)
skewed_data = np.random.exponential(10, 500)

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# QQ Plots
probplot(normal_data, dist="norm", plot=axes[0, 0])
axes[0, 0].set_title('QQ Plot — Normal Data', fontweight='bold')
axes[0, 0].get_lines()[0].set_color('#3b82f6')

probplot(skewed_data, dist="norm", plot=axes[0, 1])
axes[0, 1].set_title('QQ Plot — Skewed Data', fontweight='bold')
axes[0, 1].get_lines()[0].set_color('#ef4444')

# Histograms with KDE
sns.histplot(normal_data, kde=True, ax=axes[1, 0], color='#3b82f6')
axes[1, 0].set_title('Distribution — Normal Data')
sns.histplot(skewed_data, kde=True, ax=axes[1, 1], color='#ef4444')
axes[1, 1].set_title('Distribution — Skewed Data')

plt.tight_layout(); plt.show()

# ── Shapiro-Wilk Test ──
stat, p_val = shapiro(normal_data)
print(f"Normal data  → Shapiro p={p_val:.4f} {'✅ Normal' if p_val > 0.05 else '❌ Not Normal'}")

stat, p_val = shapiro(skewed_data)
print(f"Skewed data  → Shapiro p={p_val:.4f} {'✅ Normal' if p_val > 0.05 else '❌ Not Normal'}")
```

13.5 Log Transform for Non-Normal Data

```python
# ── Transform skewed data ──
log_data = np.log(skewed_data)

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# Original
sns.histplot(skewed_data, kde=True, ax=axes[0], color='#ef4444')
axes[0].set_title(f'Original (skew={stats.skew(skewed_data):.2f})')

# Log-transformed
sns.histplot(log_data, kde=True, ax=axes[1], color='#10b981')
axes[1].set_title(f'Log Transform (skew={stats.skew(log_data):.2f})')

# QQ Plot of transformed
probplot(log_data, dist="norm", plot=axes[2])
axes[2].set_title('QQ Plot — After Log Transform')

plt.suptitle('Transforming Skewed Data to Normal', fontsize=14, fontweight='bold')
plt.tight_layout(); plt.show()

# ── Box-Cox Transform (automatic) ──
from scipy.stats import boxcox
transformed, best_lambda = boxcox(skewed_data)
print(f"Optimal λ = {best_lambda:.4f}")
print(f"Skewness after Box-Cox: {stats.skew(transformed):.4f}")
```

14 · Quick Reference Card

🚀 Session 41 — Normal Distribution Cheat Sheet

The Bell Curve

- $f(x) = (1/\sigma\sqrt{2\pi}) \cdot e^{[-(x-\mu)^2/(2\sigma^2)]}$
- Notation: $X \sim N(\mu, \sigma^2)$
- Mean = Median = Mode = μ, Skewness = 0, Kurtosis = 3

Empirical Rule

- 68.27% within $\mu \pm 1\sigma$
- 95.45% within $\mu \pm 2\sigma$
- 99.73% within $\mu \pm 3\sigma$

Z-Score

- $Z = (X - \mu) / \sigma$ → converts any normal to $N(0,1)$
- $Z = 0$ → 50th percentile, $Z = \pm 1.96$ → 95% CI bounds

Normality Checks

- QQ Plot: Points on diagonal = normal
- Shapiro-Wilk: `stats.shapiro(data)` → $p > 0.05$ = normal
- Skewness: $|skew| < 0.5$ → approximately normal

Python Quick Reference

- `stats.norm.pdf(x, μ, σ)` — density at x
- `stats.norm.cdf(x, μ, σ)` — $P(X \le x)$
- `stats.norm.ppf(q, μ, σ)` — inverse CDF (quantile → value)
- `stats.norm.sf(x, μ, σ)` — $P(X > x)$
- `probplot(data, dist='norm', plot=ax)` — QQ plot
- `stats.shapiro(data)` — Shapiro-Wilk test
- `stats.boxcox(data)` — optimal power transform

📚 What's Next? → Other Distributions (Session 42)

The next session covers other important probability distributions: **Bernoulli, Binomial, Poisson, Uniform, Exponential** and more — each with unique properties and ML applications.